DATAOPS INNOVATION HACKATHON 2026 · PROBLEM STATEMENT 1

ARGUS

An AI security agent that investigates, verifies and explains

Adaptive Reasoning & Grounded Unified Security agent for authorized VAPT

Project plan: what we are building, how, and why it is different

Team: Rahul (RAHUL0408-B) & Pawan

Repository: github.com/RAHUL0408-B/DataOps

Hackathon window: 2 – 7 November 2026 (virtual, Discord)

Document date: 2 November 2026 · Version 1 (for team review)

1. The project in one page

What the hackathon asks for. A conversational AI agent that runs an *authorized* penetration test from start to finish: take a request and a scope, plan which tools to use and why, run (or import) the scans, merge and interpret the results, rank the risks, pause for a human before any active exploitation, and produce a client-ready PDF report. The judges explicitly say a good-looking dashboard is not enough. They want to see **real tool output, real reasoning, and every conclusion traceable to evidence**.

What we are building: ARGUS. Most teams will build a pipeline that runs Nmap → Nikto → ZAP in a fixed order and asks an LLM to summarise the output. ARGUS works like a human analyst instead. It runs an **investigation loop**: it looks at what it knows, forms hypotheses ("port 80 runs Apache 2.4.49, which may be vulnerable to path traversal"), picks the cheapest permitted check that would confirm or refute the hypothesis, runs it, and updates its beliefs. Every step is recorded as a **Decision Card** that the judges can read.

> **Our one-line pitch:** "Scanners give you alerts. ARGUS gives you conclusions: each one comes with the evidence behind it, a confidence level, and the reasoning that led to it. A human approves every risky action."

Why this should win

Judging criterion (points)	How ARGUS scores here
AI reasoning & decisions (20)	Hypothesis-driven loop. Each Decision Card shows what was observed, which options were considered, which one was chosen and why, and what was deliberately skipped.
End-to-end coverage (15)	One continuous flow: request → scope → plan → recon → enumeration → vuln scan → correlation → approval → validation → PDF.
Technical accuracy (15)	Parsers built on each tool's structured output (XML/JSON), not on screen-scraping. The LLM interprets results but never invents them.
Correlation & false positives (15)	Common finding schema, explicit merge rules, an evidence graph and a confidence ledger with reasons. Refuted findings are kept and shown, not silently deleted.
Risk prioritization (10)	Transparent formula: CVSS from the tool × exposure × exploitability (CISA KEV) × asset criticality × confidence. Every factor is visible.
Safety & scope control (10)	A scope guard written in plain code that the LLM cannot bypass, an approval gate for active actions, and a tamper-evident audit log.
Report quality (10)	Every section the brief requires, plus an evidence appendix. Each finding ID links back to its raw tool output and file hash.
Efficiency & robustness (5)	Action de-duplication, cost budget, timeouts and retries. If the LLM is unavailable, a rule-based fallback planner takes over.

2. What the rules and the brief require

2.1 Mandatory for a working prototype (we must have all of these)

- A conversational request, scope validation and an auditable plan.
- At least three working tool integrations or supported imports, **covering both network and web**.
- A common finding schema, evidence links, duplicate handling and risk ranking.
- A professional downloadable PDF report and an analyst approval gate.

2.2 Bonus features we will also target

- Multi-step adaptive replanning with state and memory (this is the core of ARGUS).
- Confidence calibration and advanced false-positive triage (the Confidence Ledger).
- Retesting and change comparison ("Retest" button that re-runs only the checks related to a finding and shows a diff).
- More tool adapters, with each one's status reported honestly.

2.3 Hard rules we must never break

Rule	What it means for our build
Test only the organizer-provided lab	The scope guard blocks any target that is not registered. During development we use only our own local Docker lab.
No uncontrolled exploitation	Metasploit runs only after an explicit, recorded human approval, and only *check* / auxiliary modules against lab hosts.
Do not fabricate scanner output	Demo replay mode is clearly labelled "REPLAY". We never present recorded runs as live.
Do not invent CVEs, proof or scores	The Grounding Validator rejects any CVE or score that does not appear in the tool output or our offline reference data.
Truthful integration matrix	Every tool is labelled Live / Import-only / Planned / Unsupported. Nessus and Burp Pro licensing is not misrepresented.
Every action attributable	Hash-chained audit log: who, what, when, why, scope decision and outcome.
Disclose AI-generated code	We will state in the README that the code was built with an AI coding assistant, and both of us must be able to explain it.
Do not leak confidential data to external AI	Scan data sent to the LLM is redacted (credentials, tokens). A local-model option is available if the organizers object.
Mark planned vs implemented	The UI and README show which features are live. Nothing is overclaimed in the demo.

2.4 What the judges will do in the live demo

The brief gives a seven-step checklist. Section 11 of this document maps each step to the exact screen and action in ARGUS, so the demo follows the judges' own list.

3. What makes ARGUS unique

We took ChatGPT's suggestions (adaptive agent, false-positive verification, attack-path view) and turned them into five concrete, buildable features. All five support the highest-weighted criteria.

1. Hypothesis-driven Investigation Loop (targets the 20-point criterion)

Fixed scan sequences are replaced by a loop: **Observe → Hypothesize → Choose action → Act → Update**. The agent keeps a list of open hypotheses, each with a status (open / supported / refuted). It picks the next action by asking which permitted check would resolve the most important open hypothesis at the lowest cost and noise. Example: it finds no web ports, so it skips ZAP and Nikto and records why. It finds WordPress, so it adds the relevant ZAP policy.

2. Decision Cards: reasoning you can audit

Each step produces a card containing the trigger (evidence IDs), the hypothesis, the candidate actions considered with the reason each was rejected, the chosen action, the expected outcome, the scope check result, and later the actual outcome. Judges can open any card and follow the reasoning.

3. Evidence Graph + Confidence Ledger (targets correlation, accuracy and FP handling)

Every finding is linked to raw evidence: tool name, file, SHA-256 hash, line or XPath reference, and timestamp. Confidence is computed by **deterministic rules with written reasons** (for example, "+25: independently reported by OpenVAS and Nikto", "−30: version-only check and the distro may backport fixes"), not by the LLM's opinion. Findings move through the states Observed → Suspected → Corroborated → Validated, or are marked Refuted (false positive, with the reason).

4. Grounding Validator: an anti-hallucination gate

Everything the LLM writes, including summaries, descriptions and report text, must cite evidence IDs. Before anything is shown or reported, a validator checks that each cited ID exists, each CVE mentioned appears in tool output or our offline CVE snapshot, and each score comes from a tool. Failing text is rejected and regenerated. The report visually separates **Observed evidence** from **AI inference**, which the brief asks for explicitly.

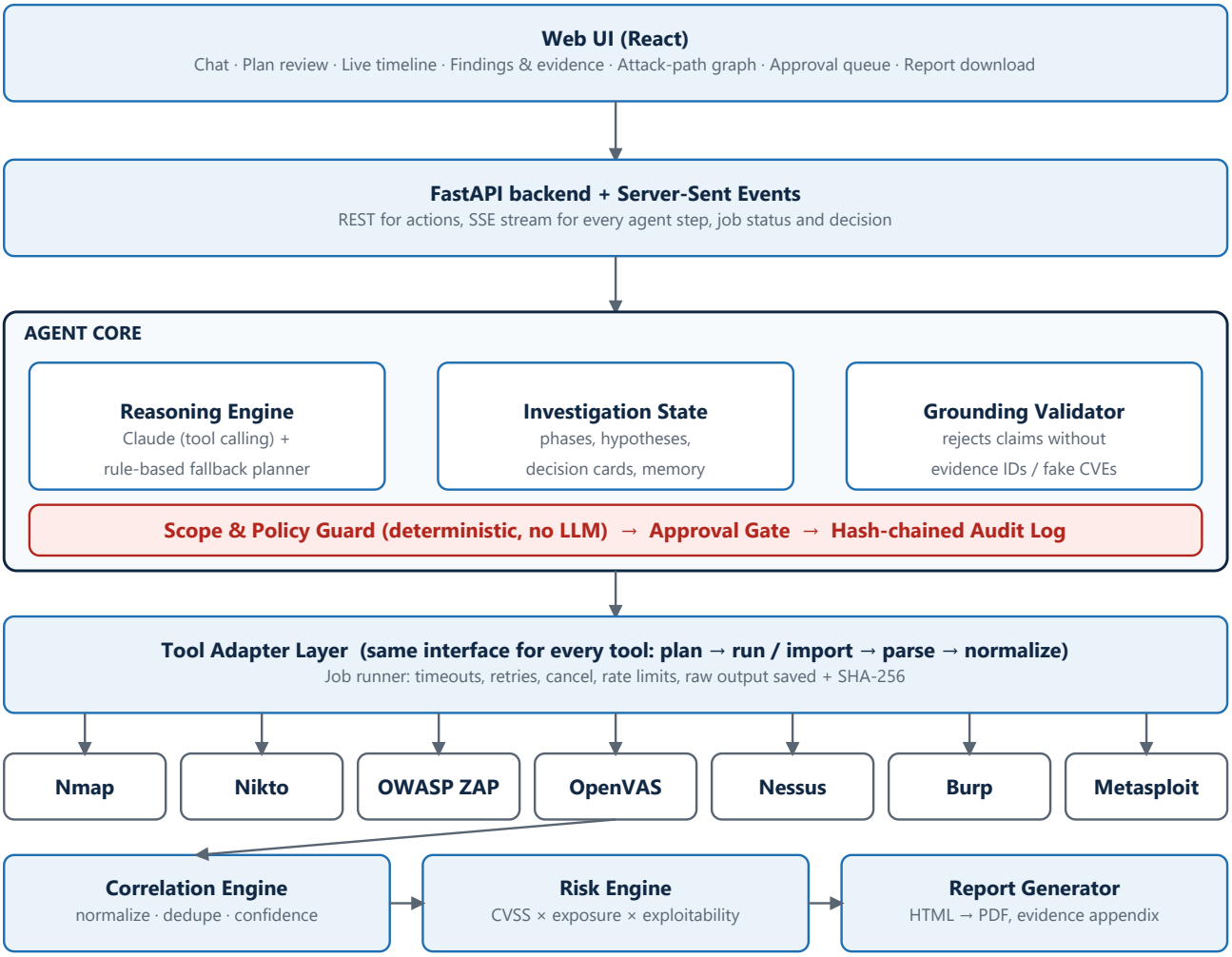
5. Attack-Path Graph with verified vs hypothesised links

An interactive graph shows how weaknesses chain together (exposed service → outdated version → known exploit → sensitive data). Solid edges are backed by evidence and dashed edges are AI hypotheses, so the uncertainty stays visible. Paths feed back into risk ranking.

Supporting differentiators (cheap to build, strong in the demo)

- **Scope guard outside the LLM:** even if the model is prompt-injected by a malicious banner or web page, it cannot touch an out-of-scope host. We will demo exactly this attack.
- **Retest & diff:** after a fix, re-run only the checks behind one finding and show whether it is Fixed, Still present or Changed.
- **Analyst copilot chat:** ask "why is F-007 high?" or "show me the evidence for this" and get answers grounded in the stored evidence.
- **Graceful degradation:** if a tool fails or times out, the agent records it, explains the coverage gap, chooses an alternative, and lists the gap under Limitations in the report.

4. Architecture



Component	Responsibility
Web UI	Chat for requests, plan approval screen, live timeline (SSE), findings table, evidence viewer, attack-path graph, approval queue, report download.
Reasoning Engine	Claude via the Anthropic API with tool calling. It can only *propose* actions through typed tools (run_nmap, run_zap, ...). It never runs shell commands directly. A rule-based planner covers the same phases if the API is down.
Investigation State	SQLite-backed memory: phase, assets and services, hypotheses, decision cards, completed actions (for de-duplication) and budget.
Scope & Policy Guard	Plain Python. Checks target CIDR/host allow-lists, exclusions, ports, tool permissions, intensity, rate limits and time window. Default is deny.
Approval Gate	Any action classed as *active* (Metasploit, intrusive ZAP active scan, credentialed brute force) waits for an analyst to approve or reject it, with a comment. Both are logged.
Tool Adapters	One class per tool with the same interface. Each adapter saves raw output, hashes it, parses it and emits normalized findings.
Correlation Engine	Normalizes data, merges genuine duplicates, links related findings, and computes confidence with reasons.
Risk Engine	Computes the priority score from the tool's CVSS, exposure, exploitability (offline CISA KEV list), asset criticality and confidence.

Component	Responsibility
Report Generator	Jinja2 HTML templates → PDF (WeasyPrint). Contains the executive view, the technical view and an evidence appendix.
Audit Log	Append-only. Each entry stores the hash of the previous entry, so tampering is detectable. Exported with the report.

Data flow (one assessment)

Analyst request → LLM extracts the target and intent → Scope Guard validates → initial plan (Decision Cards) → analyst approves the plan → Nmap recon → the agent reads the services and opens hypotheses → adapters run (Nikto/ZAP for web, OpenVAS for infrastructure, imports for Nessus/Burp) → Correlation → Risk ranking → the agent proposes validation → **human approval** → Metasploit check → status updated → Grounding Validator → PDF report and audit trail.

5. How the agent reasons

5.1 Phases (NIST SP 800-115 style)

Phase	What happens	Typical tools
0. Scope	Parse the request, match it against registered scope, reject anything outside it, confirm the rules of engagement.	Scope Guard
1. Recon / discovery	Find live hosts, open ports, services and versions.	Nmap (-sV, safe scripts)
2. Enumeration	Look deeper at what was found: HTTP technologies, TLS, SMB/FTP banners.	Nmap NSE, Nikto
3. Vulnerability analysis	Scan for known weaknesses; import commercial results.	OpenVAS, ZAP, Nessus/Burp import
4. Correlation & triage	Merge, deduplicate, score confidence, refute false positives.	Correlation Engine
5. Validation (approved)	Safe confirmation of selected high-risk findings after human approval.	Metasploit check/aux, targeted ZAP
6. Reporting	Grounded PDF report, audit trail, limitations.	Report Generator

The phases are a guide, not a fixed script. The agent may go back (for example, a new host found by ZAP crawling goes back to recon if it is in scope) or skip ahead (no web services means no web scans, and it records the reason for skipping).

5.2 How the next action is chosen

For every candidate action the agent considers: **which hypotheses it would resolve**, **severity at stake**, **cost** (time), **noise/intrusiveness**, **whether it was already done** (de-duplication), and **whether it is permitted**. The LLM ranks the candidates and writes the rationale. Deterministic code enforces de-duplication, the budget and scope, so the model cannot loop or overstep.

5.3 Example Decision Card

```
DECISION D-004                                    phase: enumeration -> vuln analysis
Trigger:     EV-0012 (nmap: 10.10.0.5:80 http Apache httpd 2.4.49)
Hypothesis:  H-03 "Apache 2.4.49 may allow path traversal (CVE-2021-41773)"
             source of CVE: OpenVAS NVT / Nmap vulners output, NOT model memory
Options considered:
  A) Nikto web config scan        cost: low   noise: medium  resolves: H-03 partly, H-05
  B) ZAP active scan full site    cost: high  noise: high    resolves: H-03, H-06  -> needs approval
  C) OpenVAS targeted NVT check   cost: med   noise: low     resolves: H-03 (version + test)
Chosen:      C, then A
Why:         C confirms H-03 with low noise; A adds config issues; B deferred until
             passive results show forms/params worth active testing.
Scope check: PASS (10.10.0.5 in lab-net, port 80 allowed, intensity=standard)
Outcome:     H-03 -> Corroborated (OpenVAS + version banner). Validation proposal V-01
             queued for human approval (Metasploit check module).
```

6. Findings, evidence and correlation

6.1 Common finding schema (every tool maps into this)

```
Finding {
  id: "F-007"                 title, description
  asset: {host, ip, port, protocol, service, url, parameter}
  identifiers: {cve[], cwe[], owasp_top10, attack_technique, tool_rule_ids[]}
  severity: {tool_severity, cvss_vector, cvss_score, cvss_source}   # never invented
  status: observed | suspected | corroborated | validated | refuted | fixed
  confidence: {score 0-100, level low/med/high, reasons[]}
  occurrences: [{tool, raw_ref -> Evidence.id, first_seen, last_seen}]
  risk: {priority_score, factors{...}, rank}
  remediation, references[], ai_inferences[ {text, cites[evidence ids]} ]
}
Evidence { id, tool, command/job_id, file_path, sha256, locator (line/XPath/JSON path),
           excerpt, captured_at }
```

6.2 Duplicate handling: merge only genuine duplicates

Situation	Action
Same host:port + same CVE from two tools	**Merge** into one finding. Both occurrences and both pieces of evidence are kept. Confidence goes up.
Same URL + parameter + same CWE (e.g. ZAP and Burp XSS)	**Merge**, keeping each tool's request/response evidence.
Same CVE on two different hosts	**Do not merge.** One finding per affected asset, grouped under one "issue" in the report.
Same weakness class, different URL/parameter	**Link as related.** Kept separate.
Version-only detection contradicted by another check	Marked **Suspected** or **Refuted** with the written reason. Kept in a "False positives" appendix.

6.3 Confidence Ledger: example rules

Rule	Effect
Reported by one tool using an active test (not just the version)	+40 base
Reported by one tool based only on a version/banner	+25 base
Independently reported by a second tool	+25
Version banner within the CVE's affected range (from tool data)	+15
Request/response proof captured (ZAP/Burp evidence)	+15
Distro package that may carry backported fixes (version-only)	−30
Another tool explicitly tested and did not reproduce	−25
Analyst-approved validation succeeded / failed	Validated (100) / Refuted

Scores map to Low (<40), Medium (40–69) and High (≥70). The reasons are printed next to every finding in the UI and the report.

7. Risk prioritization

The brief forbids fabricated scores. We therefore never ask the LLM for a number. The priority score is a transparent formula, and each factor is shown:

```
priority = min(10,  base_score  x  exposure  x  exploitability  x  asset_criticality  x  confidence_w)

base_score        CVSS base score reported by the tool (Nessus/OpenVAS/ZAP).
                  If the tool gives none: severity midpoint (Crit 9.5, High 8.0, Med 5.5,
                  Low 2.5) and labelled "derived from tool severity, no CVSS".
exposure          1.0 reachable from the tester's network without auth
                  0.85 requires authentication   0.7 local / internal-only
exploitability    1.3 CVE on CISA KEV list (offline snapshot)
                  1.15 public exploit / Metasploit module referenced by tool data
                  1.0 otherwise
asset_criticality analyst-set per asset: 0.8 low / 1.0 normal / 1.2 critical
confidence_w      validated 1.0 | corroborated 0.95 | suspected 0.75 | refuted -> excluded
```

Ranking also gives a bonus when a finding is part of a **verified** attack path. The UI shows a “why this rank” breakdown, and the LLM explains the business impact, citing the evidence. Findings are mapped to OWASP Top 10, CWE and MITRE ATT&CK techniques where the mapping is deterministic, and the report states that this mapping does not establish formal compliance.

8. Safety and scope control

- **Scope registration:** targets (CIDR/hosts/URLs), exclusions, allowed ports, allowed tools, max intensity, rate limit, time window and an “authorized by” field. Stored and shown in the report's Rules of Engagement section.
- **Deny by default:** every adapter call passes through `ScopeGuard.check(action)`. A failed check raises an error, the attempt is logged, and the agent is told why. Redirects or links to out-of-scope hosts found by crawlers are dropped.
- **Action classes:** passive / standard / intrusive / exploit. Intrusive and exploit actions always require approval, and exploit actions are limited to lab hosts.
- **Approval record:** who approved, when, the exact command or module and options, the analyst's comment, and the outcome. These are printed in the report's Limitations & Approvals section.
- **Prompt-injection hardening:** tool output is passed to the LLM as data, never as instructions. The model has no shell tool, only typed tools with validated arguments.
- **Data protection:** raw outputs are kept locally, secrets are redacted before reaching the LLM, and API keys come from environment variables (.env is git-ignored).
- **Kill switch:** one button stops all running jobs immediately.

9. Tool integration matrix (honest, as planned)

This is our **target** status. We will update it truthfully as we build. Anything that does not work by demo day will be relabelled.

Tool	Planned status	How	Notes / limits
Nmap	**Live**	CLI, -oX XML parsed	Service/version detection, safe NSE scripts (vulners, http-*, ssl-*).
Nikto	**Live**	CLI, JSON/XML output	Web server misconfiguration and known files.
OWASP ZAP	**Live**	ZAP daemon (Docker) + REST API	Spider + passive scan without approval; active scan only after approval.
OpenVAS / Greenbone	**Live** (stretch) / Import	GMP via python-gvm; XML import	Greenbone Community containers are heavy (feed sync can take hours). XML import is the fallback.
Metasploit	**Live, approval-gated**	msfrpcd + RPC client	Only *check* and auxiliary modules on lab hosts, after explicit approval. Never autonomous.
Nessus	**Import-only**	.nessus XML parser	Commercial product. We do not claim live API use. Essentials exports can be imported.
Burp Suite	**Import-only**	Burp issues XML export	Community edition has no scanner API. The Pro REST API is listed as Planned / not licensed.

This gives us five live integrations (two of them web) plus two honest imports. The prototype minimum is three, covering network and web.

10. Technology stack

Layer	Choice	Why
Language / backend	Python 3.11+, FastAPI, asyncio	Best ecosystem for security tools (python-gvm, pymetasploit3, zapv2). Async job runner.
AI model	Claude (Anthropic API, latest Sonnet) with tool calling	Strong reasoning and reliable structured tool calls. Model name is configurable. A rule-based fallback planner covers outages.
State	SQLite + SQLModel; artifacts folder with SHA-256	Zero setup, easy to ship, and inspectable by judges.
Frontend	React + Vite + Tailwind, Cytoscape.js (graph)	Fast to build, live updates via SSE.
Report	Jinja2 → HTML → PDF (WeasyPrint)	Professional layout, the same template drives the HTML preview and the PDF.
Runtime / lab	Docker Compose: ARGUS + Kali tools + ZAP + msfrpcd + lab targets	One command to run. Dev targets: Metasploitable 2, DVWA, OWASP Juice Shop (local, isolated network).
Reference data	Offline snapshots: CISA KEV, CWE, OWASP Top 10, ATT&CK mapping	No live lookups needed in the demo, and no invented data.

11. Live demo script mapped to the judges' checklist

#	Judge checks	What we show in ARGUS
1	Authorized target + conversational request, scope validation	Type “Assess lab-net 10.10.0.0/24, focus on web apps, no DoS”. Then ask it to also scan 8.8.8.8: the request is **rejected**, the reason is shown, and the attempt is logged.
2	Plan with tool selection and reasoning	Plan screen lists Decision Cards (chosen tools, skipped tools, and why). The analyst clicks Approve plan.
3	Multi-tool execution, real status, traceable output	Live timeline: Nmap running, then done (raw XML viewable). The agent adapts: web found, so Nikto + ZAP are added. Nessus/Burp files are imported and labelled IMPORT.
4	Normalization, correlation, duplicates, false positives, uncertainty	Findings view: a merged finding showing “2 tools”, a Refuted finding with its reason, and confidence reasons.
5	Risk ranking + human approval before active validation	“Why this rank” breakdown. The agent proposes a Metasploit check, the **approval dialog** appears, we approve, and the result moves the finding to Validated.
6	Download report, trace a finding to source evidence	Download the PDF. Pick F-00X: the evidence ID leads to the raw file, line and SHA-256 hash in the Evidence appendix and in the UI.
7	Safe failure handling, unsupported integrations	Kill the ZAP container mid-run: the agent records the failure, retries, then reports the coverage gap. The integration matrix page shows Burp Pro as Unsupported.

12. Build plan (2 – 7 November)

Day	Deliverable	Done when
Day 1 – Nov 2	Repo skeleton, Docker lab (targets + tools), scope guard, finding schema, Nmap adapter	A scope-checked Nmap scan stores raw XML and normalized assets.
Day 2 – Nov 3	Nikto + ZAP adapters, Nessus/Burp importers, job runner (timeouts/retries), audit log	Three or more tools produce normalized findings with evidence hashes.
Day 3 – Nov 4	Agent loop with Claude tool calling, Decision Cards, hypotheses, fallback planner	The agent adapts its plan to discovered services and explains each step.
Day 4 – Nov 5	Correlation + confidence ledger, risk engine, grounding validator, approval gate + Metasploit	Merged and refuted findings are visible; approval is recorded.
Day 5 – Nov 6	React UI (chat, timeline, findings, graph, approvals), PDF report	A full run produces a downloadable report end to end.
Day 6 – Nov 7	OpenVAS (stretch), retest/diff, hardening, docs, architecture note, matrix, demo rehearsal	Rehearsed demo; all deliverables submitted.

Suggested split: I (Claude, working in Rahul's repo) build the core code. Rahul reviews, runs it, and keeps decisions on track. Pawan takes the parts that need a human: setting up and testing the Docker lab on a real machine, obtaining Nessus/Burp sample exports, recording and rehearsing the demo, and checking the organizer's Discord for the official lab target details.

13. Required deliverables checklist

Deliverable (brief §9)	Where it comes from
9.1 Working agent against an organizer-authorized lab	ARGUS + Docker Compose. The target is set in scope config once organizers share it.
9.2 Architecture overview (model, orchestration, adapters, data flow, state, boundaries)	docs/architecture (expanded from section 4 of this plan).
9.3 Sample AI-generated VAPT report (PDF)	Generated by ARGUS from a real lab run, committed to the repo.
9.4 Repo with setup, dependencies, licensing assumptions, execution steps	README (written at the end, as you asked).
9.5 Human-approval demo + decision/audit trace	Approval dialog plus exported audit log (JSON + report appendix).
9.6 Integration status matrix with limitations / assumptions / unsupported scenarios	Section 9, kept up to date and also shown in the UI.

14. Decisions we need from you

Please confirm or change these before I start building:

#	Question	My recommendation
1	Project name	**ARGUS** (the hundred-eyed watchman of Greek myth)
2	LLM provider. Do you have an Anthropic API key (or credits)? If not, which key do you have?	Claude (Anthropic API). A rule-based fallback keeps the demo working without a key.
3	Can we run Docker (Docker Desktop + WSL2) on your machine? The tools (Nmap, Nikto, ZAP, Metasploit) run best in Linux containers.	Yes, run everything in Docker.
4	Have the organizers shared the lab target yet (IP/URL, credentials, rate limits)?	Until they do, we build against our own local lab (Metasploitable 2, DVWA, Juice Shop).
5	OpenVAS: attempt a live integration (heavy, slow feed sync) or import-only?	Try live on Day 6, keep import as a guaranteed fallback.
6	Frontend: React (more polished) or Streamlit (faster, plainer)?	React; Streamlit as a fallback if time runs short.
7	Git commits: authored as RAHUL0408-B <rahulbramhankar04@gmail.com>, no Claude co-author line. Is this the right email?	Yes. This is the git identity already configured on this machine.

> Once you reply with your changes, I will start with Day 1: the repo skeleton, Docker lab, scope guard, finding schema and the Nmap adapter. I will commit and push after every completed task.